WHITEPAPER

libp2p-raft Architecture

RaftEngine, RaftBehaviour, RaftHandler
and rust-libp2p Swarm integration

HoodLabs — libp2p-raft

Document version 2.0 | November 2, 2026

Detailed technical document for research and experimentation.
Not a substitute for production operations documentation.

Abstract

This whitepaper describes the **complete architecture** of the `libp2p-raft` crate: how a Raft state machine is embedded into rust-libp2p through the `NetworkBehaviour` / `ConnectionHandler` model, module boundaries, runtime data structures, event contracts, election/replication/inbound-outbound RPC flows, the wire specification, the failure matrix, storage and snapshots, and the event-driven execution model. The content tracks the source in `src/behaviour.rs`, `src/handler.rs`, `src/raft/engine.rs`, `src/protocol/` and `src/storage/`.

Keywords: Raft, libp2p, Swarm, NetworkBehaviour, ConnectionHandler, unary RPC, Yamux, hexagonal architecture, correlation ID, snapshot.

Contents

1 Introduction 4

2 Raft background (structured summary) 4
- 2.1 Roles and time . . . 4
- 2.2 Safety . . . 4
- 2.3 Persistent and volatile state . . . 4

3 rust-libp2p background 5
- 3.1 Swarm . . . 5
- 3.2 Unary RPC over Yamux . . . 5
- 3.3 Hexagonal / ports-and-adapters . . . 5

4 Layered architecture overview 5
- 4.1 Infrastructure architecture . . . 5
- 4.2 Responsibilities . . . 7
- 4.3 What libp2p provides vs. what we write . . . 7
- 4.4 Byte stack (application down to the wire) . . . 7
- 4.5 Invariant along the consensus path . . . 7

5 Execution and threading model 7

6 RaftEngine — a pure state machine 8
- 6.1 Public API . . . 8
- 6.2 Action types . . . 8
- 6.3 Key internal state . . . 8
- 6.4 The `tick` flow . . . 8
- 6.5 Replication and commit . . . 9
- 6.6 RequestVote and AppendEntries (logic) . . . 9

7 RaftBehaviour — the NetworkBehaviour adapter 9
7.1 Networking state . . . 9
7.2 NetworkBehaviour callbacks . . . 10
7.3 `execute_actions` . . . 10
7.4 `send_rpc` . . . 10
7.5 Public events for the application . . . 10

8 `RaftBehaviour::poll` ordering 10

9 RaftHandler — ConnectionHandler 11
9.1 FromBehaviour / ToBehaviour contract . . . 11
9.2 Handler state . . . 11
9.3 `RaftHandler::poll` ordering . . . 11
9.4 Inbound unary (detailed) . . . 11
9.5 Outbound unary . . . 11

10 Protocol and codec specification 11
10.1 Protocol ID and framing . . . 11
10.2 WireEnvelope and RaftMessage . . . 12
10.3 Codec (`protocol/codec.rs`) . . . 12

11 Identity and bootstrap 12

12 End-to-end control flows 12
12.1 Inbound RPC (peer → us) . . . 12
12.2 Outbound RPC (us → peer) . . . 13
12.3 Election . . . 14
12.4 Propose → commit . . . 14

13 Failure matrix and edge cases 15

14 Storage and snapshots 15
14.1 The `Storage` contract . . . 15
14.2 Data model . . . 16
14.3 The two existing backends . . . 16
14.4 Local compaction . . . 16
14.5 Leader → follower snapshot transfer . . . 17
14.6 Recovery on startup . . . 17
14.7 Production risks . . . 18
14.8 Production recommendations . . . 18

14.8.1 P0 — Durability and safety (mandatory) 18
14.8.2 P1 — Non-blocking snapshots and streaming transfer 19
14.8.3 P2 — Performance and operations 19

15 Module map of the crate 20

16 Deliberate limitations and status 20

17 Conclusion 20

1 Introduction

A replicated state machine needs a consensus protocol so that several processes agree on a single ordered history of commands, even in the presence of crashes or temporary network partitions. Raft [1] decomposes the problem into leader election, log replication and safety invariants, and is comparatively easier to implement and test than Paxos.

In parallel, libp2p [2] provides a modular networking stack: transport upgrades, cryptographic identity, and substream multiplexing. In rust-libp2p an application expresses a protocol through `NetworkBehaviour` (one instance per node) and `ConnectionHandler` (one instance per connection) rather than through raw sockets.

libp2p-raft is a research implementation that plugs a mini-Raft into a `NetworkBehaviour`. The crate does **not** own the `Swarm`: the host (`bin/raft-node.rs`) assembles TCP + Noise + Yamux via `SwarmBuilder`, attaches `RaftBehaviour`, and drives the event loop.
Architectural goals:

(i) make the Behaviour–Handler lifecycle and the unary-RPC-over-substream model explicit;
(ii) keep consensus logic independent of `PeerId`, dialing and stream handles — the engine is testable without a Swarm.

This document goes deep into module boundaries, event contracts, control flows and the execution model. It does not attempt to prove that the Raft implementation is production-ready.

2 Raft background (structured summary)

2.1 Roles and time

Time is divided into monotonically increasing *terms*. Each server is in one of three roles: **Follower** (passive), **Candidate** (requesting votes), **Leader** (accepting commands, replicating). A follower whose `election_timeout` expires becomes a Candidate → `RequestVote`. The leader sends empty `AppendEntries` as heartbeats to reset follower timeouts.

2.2 Safety

- **Election safety:** at most one valid leader per term (majorities intersect).
- **Log matching:** check `prev_log_index` / `prev_log_term`; the leader decrements `next_index` on reject.
- **State machine safety:** the same log prefix is applied on every replica.

2.3 Persistent and volatile state

Hard state: `current_term`, `voted_for`, the log. **Volatile:** `commit_index`, `last_applied`; on the leader additionally `next_index`, `match_index`. In libp2p-raft the `Storage::persist` contract describes an atomic batch; implementations may be in-memory or file-based — durability depends on the backend (see Section 14).

3 rust-libp2p background

3.1 Swarm

The `Swarm` is the node-level event loop: transports, listeners, dialing, connection pool. Each connection runs in its own task holding one `ConnectionHandler`. Behaviour and handler do **not** call each other directly — the Swarm routes `NotifyHandler` and `on_connection_handler_event`.

3.2 Unary RPC over Yamux

Long-lived connections (TCP + Noise + Yamux) amortise handshake cost. Each Raft RPC opens a new **substream**, negotiates `/libp2p-raft/1.0.0`, exchanges one request and one response, and closes the substream. Concurrent RPCs do not block each other on the same connection.

3.3 Hexagonal / ports-and-adapters

The engine takes: ticks (`Instant`), RPCs (`NodeId`, `RaftMessage`), proposals. It returns `Actions` (send to node k, apply, change role). The adapter (`RaftBehaviour`) handles identity mapping, correlation IDs, dialing and timeouts — separating *loss of connectivity* (a network event) from *membership change* (a change through the log).

Layer boundary rule

Engine → only `NodeId`, `RaftMessage`, `Instant`.
Behaviour → `NodeId` ↔ `PeerId`, `correlation_id`, RPC lifecycle, dialing.
Handler → `WireEnvelope` bytes on a substream.

4 Layered architecture overview

4.1 Infrastructure architecture

Building a Raft protocol on libp2p takes two infrastructure views: what runs *inside* one node (Figure 1) and how nodes are *deployed and connected* as a cluster (Figure 2).

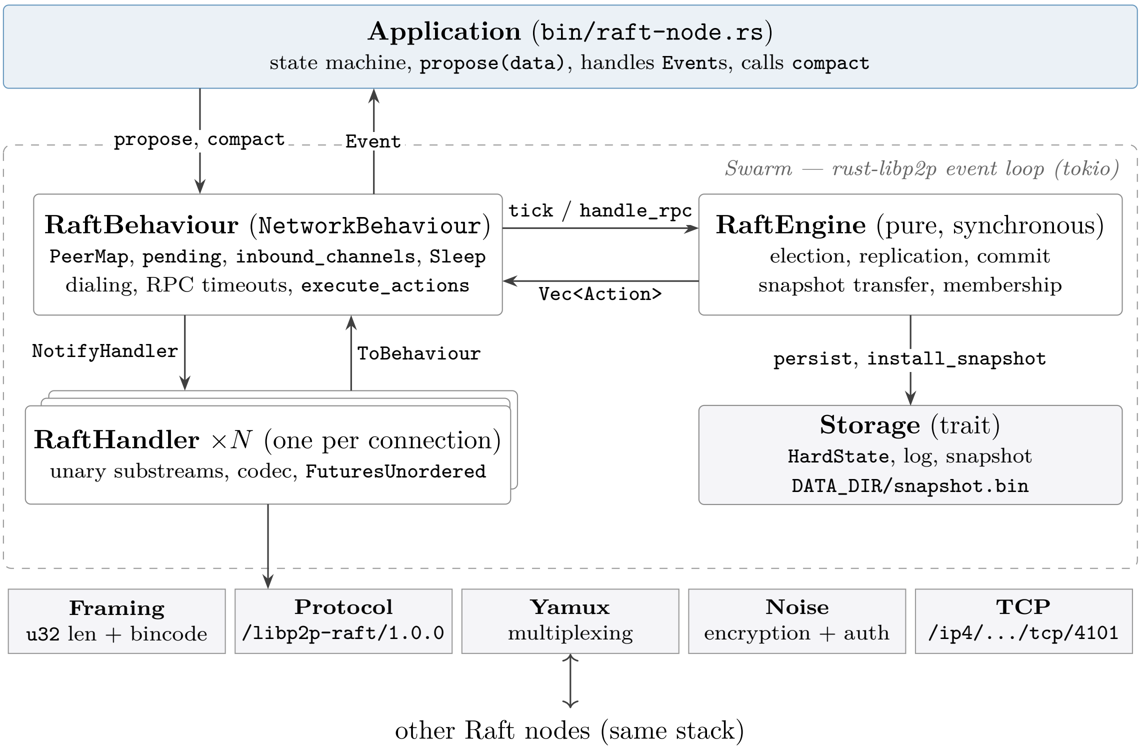
Application (bin/raft-node.rs)
state machine, propose(data), handles Events, calls compact
propose, compact
Event
Swarm — rust-libp2p event loop (tokio)
RaftBehaviour (NetworkBehaviour)
PeerMap, pending, inbound_channels, Sleep
dialing, RPC timeouts, execute_actions
tick / handle_rpc
Vec<Action>
RaftEngine (pure, synchronous)
election, replication, commit
snapshot transfer, membership
NotifyHandler
ToBehaviour
persist, install_snapshot
RaftHandler ×N (one per connection)
unary substreams, codec, FuturesUnordered
Storage (trait)
HardState, log, snapshot
DATA_DIR/snapshot.bin
Framing
u32 len + bincode
Protocol
/libp2p-raft/1.0.0
Yamux
multiplexing
Noise
encryption + auth
TCP
/ip4/.../tcp/4101
other Raft nodes (same stack)

Figure 1: Inside one node: the Raft engine is pure logic owned by the behaviour; all networking goes through the Swarm, the per-connection handlers and the libp2p transport stack.

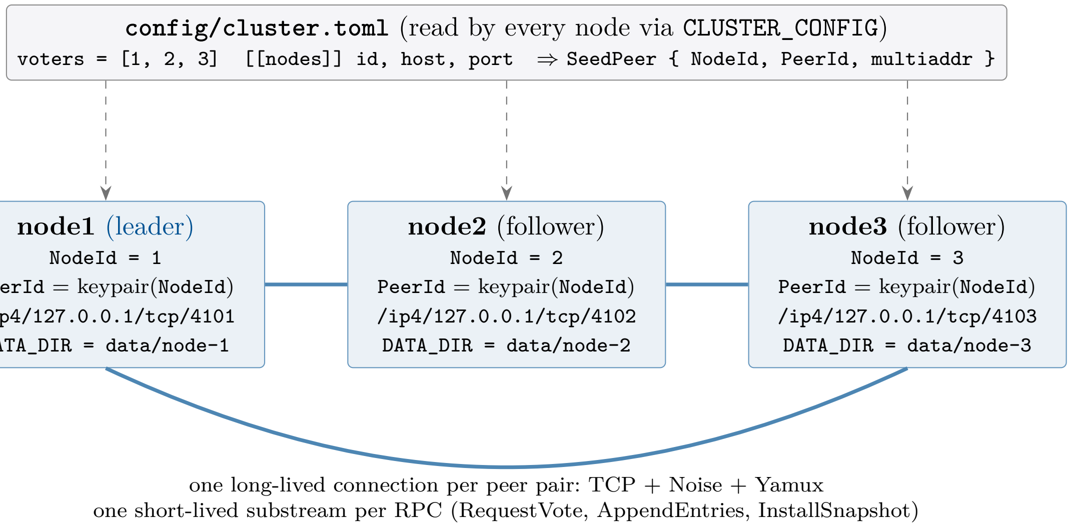
config/cluster.toml (read by every node via CLUSTER_CONFIG)
voters = [1, 2, 3] [[nodes]] id, host, port ⇒ SeedPeer { NodeId, PeerId, multiaddr }
node1 (leader)
NodeId = 1
PeerId = keypair(NodeId)
/ip4/127.0.0.1/tcp/4101
DATA_DIR = data/node-1
node2 (follower)
NodeId = 2
PeerId = keypair(NodeId)
/ip4/127.0.0.1/tcp/4102
DATA_DIR = data/node-2
node3 (follower)
NodeId = 3
PeerId = keypair(NodeId)
/ip4/127.0.0.1/tcp/4103
DATA_DIR = data/node-3
one long-lived connection per peer pair: TCP + Noise + Yamux
one short-lived substream per RPC (RequestVote, AppendEntries, InstallSnapshot)

Figure 2: Cluster deployment (3 voters, `config/cluster.local.toml`). The Docker setup (`docker-compose.yml`) runs the same binary as 6 nodes on a bridge network `172.28.0.11-16`, ports `4101-4106`; only `NODE_ID` differs per container.

4.2 Responsibilities

Layer	File / type	Responsibility
Application	`bin/raft-node.rs`	`SwarmBuilder`, `listen_on`, dial seeds, `propose`, snapshot compaction, `select_next_some`
Swarm	libp2p	Transport, pool, polling behaviour + handlers
`RaftBehaviour`	`behaviour.rs`	Engine, `PeerMap`, `pending`, `inbound_channels`, `Sleep`, `execute_actions`
`RaftHandler`	`handler.rs`	Substreams, codec, concurrent I/O
`RaftEngine`	`raft/engine.rs`	Election, replication, commit, membership logic
`Storage`	`storage/`	Hard state, log, snapshot API
Protocol	`protocol/`	`RaftMessage`, `WireEnvelope`, codec, upgrade
`PeerMap`	`peer_map.rs`	`NodeId` ↔ `PeerId`, seed multiaddrs

4.3 What libp2p provides vs. what we write

Only **four** pieces are implemented for Raft networking: (1) `RaftBehaviour`, (2) `RaftHandler`, (3) protocol + codec, (4) `PeerMap`. Everything else is libp2p: Swarm, TCP/Noise/Yamux, multistream-select.

4.4 Byte stack (application down to the wire)

Layer	Content
Raft logic	`RaftMessage` inside `WireEnvelope`
Framing	u32 BE length + bincode payload
Substream	multistream-select `/libp2p-raft/1.0.0`
Multiplex	Yamux
Security	Noise (`PeerId`)
Transport	TCP (multiaddr)

4.5 Invariant along the consensus path

Along `propose` → replicate → commit, the message shape stays `RaftMessage` from the engine to bincode; only the behaviour/handler add `correlation_id` and framing. Storage does not influence the wire shape.

5 Execution and threading model

- **One** Swarm event loop (tokio) per process.
- `RaftEngine` is called **synchronously** from `poll`, `on_connection_handler_event` and `propose` — there is no separate consensus thread.
- The handler polls `FuturesUnordered` — substream I/O runs concurrently within the same connection task.
- Engine deadlines are absolute `Instant`s; the behaviour calls `arm_sleep(min(engine, rpc_timeouts))`.

6 RaftEngine — a pure state machine

6.1 Public API

`RaftEngine<S: Storage>` does not import libp2p. Its synchronous surface:

- `tick(now)` — election or heartbeat deadline; returns `TickOutcome { actions, next_deadline }`.
- `handle_rpc(from, msg, now)` — process an inbound RPC; returns `Vec<Action>`.
- `handle_rpc_failure(to, kind)` — clears `ae_inflight` when an AppendEntries RPC fails (and aborts an in-progress snapshot transfer).
- `propose(data)` — leader appends an entry; returns the index + replication actions.
- `compact(index, blob)` — store a snapshot and compact the log prefix.
- `next_deadline()`, `role()`, `commit_index()`, . . .

6.2 Action types

Action	Meaning
`Send { to, msg }`	Send an RPC to one `NodeId`
`Broadcast { msg }`	Behaviour fans out to every other voter
`Apply { entries }`	Committed entries — apply to the state machine
`BecomeLeader` / `Follower` / `Candidate`	Role change (event for the app)
`SnapshotNeeded` / `SnapshotInstallComplete`	Snapshot hooks (app + storage)

6.3 Key internal state

- `role`, `leader`, `votes` — election.
- `next_index`, `match_index` — leader replication per follower.
- `ae_inflight: HashSet<NodeId>` — **pipeline depth 1**: no further AppendEntries is sent to a peer until a response or a failure arrives.
- `last_ae` — remembers the most recently sent prefix (for reject handling).
- `election_deadline`, `heartbeat_deadline` — absolute `Instant`s.
- `membership` — the voter set; quorum $\lfloor n/2 \rfloor + 1$.
- `snap_out`, `snap_in` — outbound/inbound snapshot transfer state.

6.4 The `tick` flow

Follower/Candidate: if `now >= election_deadline` → `start_election`: increment term, persist the self-vote, `Broadcast RequestVote`.

Leader: if `now >= heartbeat_deadline` → `replicate_actions()` (AppendEntries to each voter via `send_append_entries_to`), reset the heartbeat deadline.

`next_deadline()` = min(heartbeat, election) on the leader; followers only have the election deadline.

6.5 Replication and commit

`build_append_entries(to)` builds `prev_log_*` and the entry slice from `next_index[to]`. Success → update `match_index`; reject → decrement `next_index` (no conflict-index hint). If `next_index[to]` falls below `first_index`, the leader switches to `InstallSnapshot`. `maybe_commit` scans from high to low: only entries from the leader's **current term** may be committed by a majority `match_index`.

6.6 RequestVote and AppendEntries (logic)

Inbound `RequestVote`: reject if `term < current_term`; if `term > current` → `become_follower`; grant if not yet voted (or voted for the same candidate) and the candidate's log is up to date. Always reply with `RequestVoteResp` via `Action::Send { to: from, ...}`.

Inbound `AppendEntries`: check term and log consistency, append entries, update `commit_index` from `leader_commit`, reply with `AppendEntriesResp`.

The engine knows nothing about the network

Every `Action::Send` carries only a `NodeId`. The behaviour decides about dialing, `PeerId`s and unary substreams.

7 RaftBehaviour — the NetworkBehaviour adapter

7.1 Networking state

Field	**Role**
`engine`	The pure `RaftEngine`
`peer_map`	Static map built from `seed_peers`
`connected`	`PeerId` → `Set<ConnectionId>`
`dialing`	Avoids dial spam
`dial_backoff_until`	Backoff after transport errors
`pending`	`correlation_id` → `PendingRequest` (outbound awaiting a response)
`inbound_channels`	`channel_id` → `InboundChannel` (inbound reply)
`pending_events`	Queue of `ToSwarm`
`sleep`	Timer until `earliest_wake`

`PendingRequest` stores `to`, `peer`, `kind` (`RequestVote` / `AppendEntries` / snapshot), `sent_at`, and a copy of `msg`.

7.2 NetworkBehaviour callbacks

Method	**RaftBehaviour**
`handle_pending_outbound_connection`	Returns multiaddrs from `PeerMap` for dialing
`handle_established_*_connection`	Returns `RaftHandler::new()`
`on_swarm_event`	Updates `connected/dialing`; connection closed → `fail_peer_pending`
`on_connection_handler_event`	Dispatches `ToBehaviour::Request/Response/Failure`
`poll`	Drains events, RPC timeouts, `tick`, `Sleep`

7.3 `execute_actions`

Loop over each `Action`:

- `Send`: if `inbound_from = Some((from, channel_id))` and `to == from` → `SendResponse` + `NotifyHandler::One(conn)`; otherwise `send_rpc`.
- `Broadcast`: `send_rpc` to each of `other_voters()`.
- Role / Apply / Snapshot → `GenerateEvent`.

7.4 `send_rpc`

1. `peer_map.peer_id(to)` — unknown → `handle_rpc_failure`, `RpcFailed`.
2. Not `is_connected(peer)` → failure + `dial_seed` (lossy; no RpcFailed is emitted on the AE retry path).
3. Allocate a `correlation_id`, `pending.insert`, queue `NotifyHandler::Any` + `SendRequest`.

7.5 Public events for the application

`RoleChanged`, `Committed`, `PeerMapped` (startup), `SnapshotNeeded`, `SnapshotInstalled`, `RpcFailed`.

8 `RaftBehaviour::poll` ordering

B1. Pop `pending_events` → `Poll::Ready(ToSwarm)`.
B2. Scan `pending` for entries past `rpc_timeout` → `fail_pending` → engine `handle_rpc_failure`.
B3. If `now >= engine.next_deadline()` → `tick` + `execute_actions`.
B4. Poll `Sleep`; Ready → re-arm `earliest_wake` and loop; Pending → return.

The next wake-up deadline:

```
earliest_wake = min(
    engine.next_deadline(),
    min_pending(sent_at + rpc_timeout),
)
```

`pending_events` are drained **first** so that Dial/NotifyHandler/Event are not starved under load.

9 RaftHandler — ConnectionHandler

9.1 FromBehaviour / ToBehaviour contract

Direction	**Variant**
BH → H	`SendRequest { correlation_id, msg }`
BH → H	`SendResponse { channel_id, correlation_id, msg }`
H → BH	`Request { correlation_id, msg, channel_id }`
H → BH	`Response { correlation_id, msg }`
H → BH	`Failure { correlation_id?, error }`

9.2 Handler state

`pending_outbound`, `inflight` (`FuturesUnordered`), `response_txs` (oneshot for inbound writes), `pending_events`, and the `inbound_channel_id` allocator.

`connection_keep_alive() == true` — keeps the connection open between unary heartbeats.

9.3 `RaftHandler::poll` ordering

H1. Drain `pending_events` → `NotifyBehaviour`.
H2. Pop `pending_outbound` → `OutboundSubstreamRequest` + `OutboundInfo`.
H3. Poll `inflight`: `OutboundDone`, `InboundRead`, `InboundWritten`, `Failed`.

9.4 Inbound unary (detailed)

1. `FullyNegotiatedInbound` → asynchronously read the frame.
2. `InboundRead`: insert `response_txs[channel_id]`, queue `ToBehaviour::Request`.
3. Spawn a write future that waits on the oneshot for `SendResponse`.
4. Encode the response → `InboundWritten`.

9.5 Outbound unary

1. `SendRequest` → queue outbound.
2. Negotiate the outbound substream.
3. Write the request frame, read the response.
4. Return `ToBehaviour::Response` to the behaviour.

10 Protocol and codec specification

10.1 Protocol ID and framing

- Protocol: `/libp2p-raft/1.0.0` (`ReadyUpgrade<StreamProtocol>`).
- Frame: `u32` big-endian length + bincode(`WireEnvelope`).
- `MAX_FRAME_BYTES` = 4 MiB in the handler.

10.2 WireEnvelope and RaftMessage

```
pub struct WireEnvelope {
    pub correlation_id: u64,
    pub msg: RaftMessage,
}
```

Variants: `RequestVote`, `RequestVoteResp`, `AppendEntries`, `AppendEntriesResp`, `InstallSnapshot`, `InstallSnapshotResp`. Heartbeat = `AppendEntries` with `entries: []`.

10.3 Codec (`protocol/codec.rs`)

- `encode_envelope`: bincode-serialize, prepend the length.
- `decode_envelope`: `Cursor::new(bytes)` reads the `u32`, slices the payload, deserializes — entirely in memory.

11 Identity and bootstrap

`NodeId` (u64) = Raft identity / membership. `PeerId` = libp2p keypair (must be pinned). `SeedPeer { node_id, peer_id, addrs }` lives in the config. Voters are configured before the first election timeout. No DHT is needed for a minimal deployment: dial seeds or accept inbound connections + a shared `PeerMap`.

`node_identity.rs`: deterministic keypair per `node_id` in the demo (a real deployment must keep keys stable).

12 End-to-end control flows

The four sequence diagrams below share one convention: each column is a component, a solid arrow is a call or message, a dashed arrow is a return value or response, and a grey box is local processing.

12.1 Inbound RPC (peer → us)

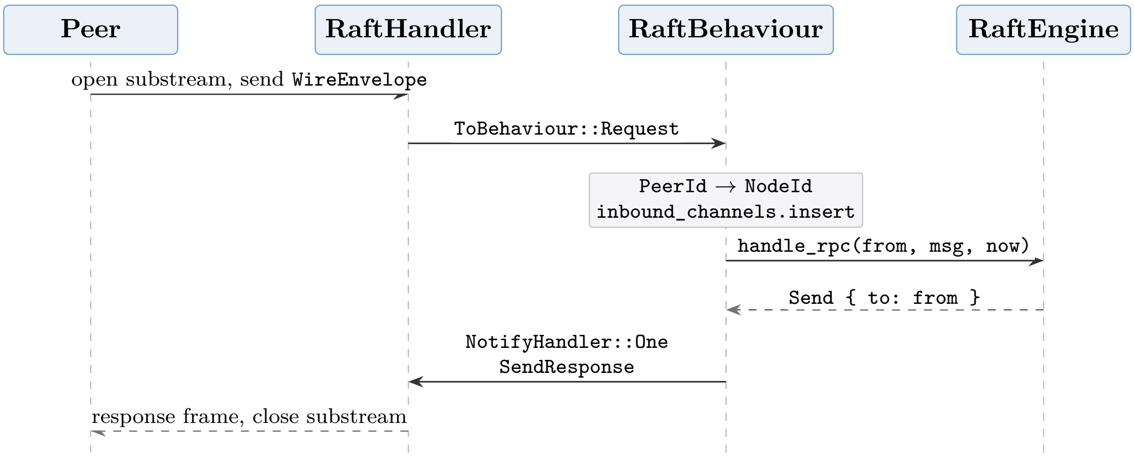
Peer
RaftHandler
RaftBehaviour
RaftEngine
open substream, send WireEnvelope
ToBehaviour::Request
PeerId → NodeId
inbound_channels.insert
handle_rpc(from, msg, now)
Send { to: from }
NotifyHandler::One
SendResponse
response frame, close substream

Figure 3: Inbound RPC: the reply travels on the *same* substream that carried the request.

1. The peer opens an inbound substream; the handler reads the `WireEnvelope`.

2. `ToBehaviour::Request` + `channel_id`; a oneshot waits for the response.
3. Behaviour: `PeerId` → `NodeId`; `inbound_channels.insert`.
4. `engine.handle_rpc(from, msg, now)` → actions (usually a `Send` back to `from`).
5. `execute_actions` with `Some((from, channel_id))` → `SendResponse` + `NotifyHandler::One`.
6. The handler writes the frame and closes the unary substream.
7. If the engine does not reply to `from`: the behaviour calls `remove(channel_id)` to avoid a leak.

12.2 Outbound RPC (us → peer)

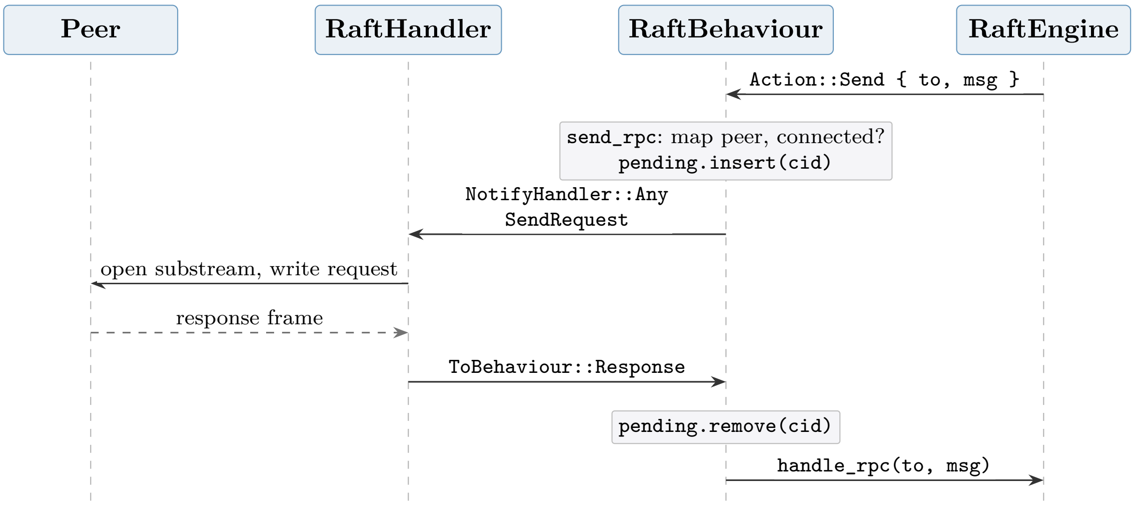
Peer
RaftHandler
RaftBehaviour
RaftEngine
Action::Send { to, msg }
send_rpc: map peer, connected?
pending.insert(cid)
NotifyHandler::Any
SendRequest
open substream, write request
response frame
ToBehaviour::Response
pending.remove(cid)
handle_rpc(to, msg)

Figure 4: Outbound RPC: the response is matched to its request by `correlation_id` (cid).

1. The engine emits `Action::Send`, or a `Broadcast` is fanned out.
2. `send_rpc`: map the peer, check connectivity, `pending.insert(correlation_id)`.
3. `NotifyHandler::Any` + `SendRequest`.
4. The handler opens an outbound substream, writes and reads.
5. `ToBehaviour::Response` → `pending.remove` → `handle_rpc(pending.to, msg)`.

Not connected: dial + `handle_rpc_failure` (lossy); the election/heartbeat tick retries later.

12.3 Election

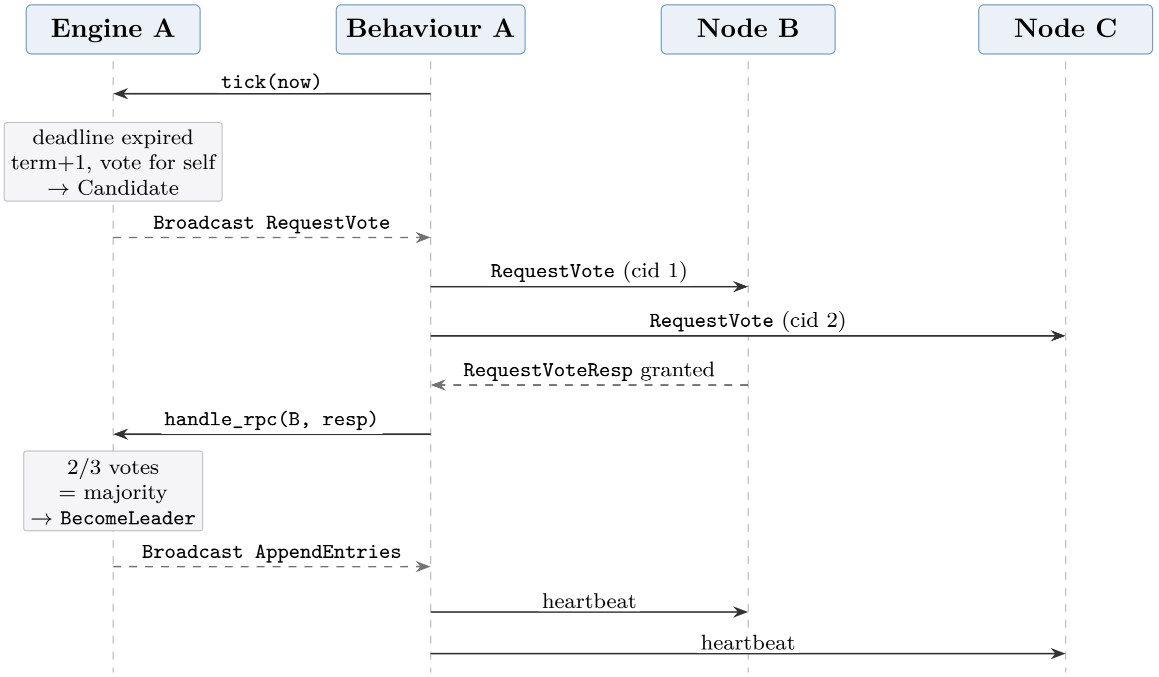
Engine A
Behaviour A
Node B
Node C
tick(now)
deadline expired
term+1, vote for self
→ Candidate
Broadcast RequestVote
RequestVote (cid 1)
RequestVote (cid 2)
RequestVoteResp granted
handle_rpc(B, resp)
2/3 votes
= majority
→ BecomeLeader
Broadcast AppendEntries
heartbeat
heartbeat

Figure 5: Election in a 3-node cluster: A wins with its own vote and B's vote.

1. A follower's timeout expires → `tick` → it becomes a candidate.
2. `Broadcast RequestVote` → the behaviour calls `send_rpc` N times.
3. Responses are matched by `correlation_id`.
4. Majority reached → `BecomeLeader`, initialise `next_index`, start heartbeats.

12.4 Propose → commit

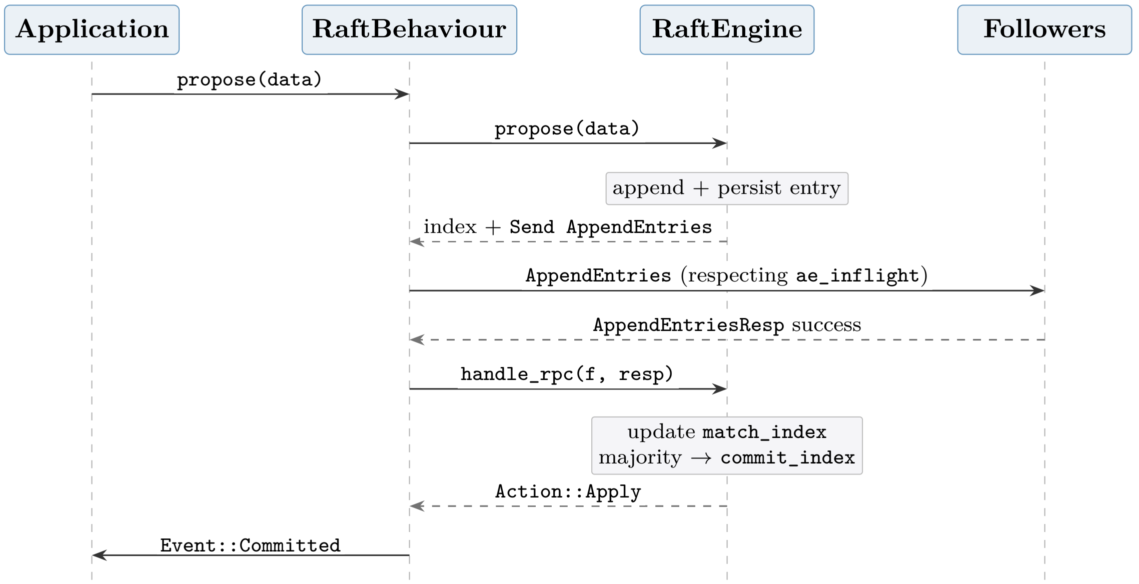
Application
RaftBehaviour
RaftEngine
Followers
propose(data)
propose(data)
append + persist entry
index + Send AppendEntries
AppendEntries (respecting ae_inflight)
AppendEntriesResp success
handle_rpc(f, resp)
update match_index
majority → commit_index
Action::Apply
Event::Committed

Figure 6: Propose → commit: an entry is applied once a majority has replicated it.

1. The app calls `propose` → the engine persists the entry.

2. The engine emits replication actions → AppendEntries per follower (respecting `ae_inflight`).
3. `AppendEntriesResp` success → update `match_index`.
4. Majority reached → advance `commit_index`, `Apply` → `Event::Committed`.

13 Failure matrix and edge cases

Situation	Behaviour
Not connected on Send	Dial + `handle_rpc_failure`; AE in-flight cleared; tick retries
RPC timeout	`fail_pending`; that RPC is not retried directly
Connection closed	`fail_peer_pending` for the peer
Dial failure	Backoff + `RpcFailed` (depending on the path)
Unknown `PeerId`	`RpcFailed`; the engine is not called
Stale term	Engine ignores / steps down per Raft rules
Multiple connections to one peer	Inbound reply must use `One(conn)`; outbound `Any` picks one
Out-of-order responses	Matched by `correlation_id`; no FIFO assumption
Connection drop	Voter is **not** removed; quorum follows the config
Membership change	Only through the log (planned; `EntryType` currently has only `Command`)

14 Storage and snapshots

This section details the storage layer and the snapshot mechanism as implemented in `src/storage/` and `src/raft/snapshot.rs`, identifies the risks of running it in production, and proposes a redesign ordered by priority.

14.1 The `Storage` contract

The engine accesses persistence only through the `Storage` trait (generic `RaftEngine<S: Storage>`), so backends can be swapped without touching Raft logic.

Method	Meaning and constraints
`hard_state()`	Current `current_term` and `voted_for`.
`first_index()`	First index still in the log: `last_included_index + 1`, or 1 if there is no snapshot.
`entry(i)`	Entry at index `i`; `None` if compacted away or not yet present.
`last_index_term()`	`(index, term)` of the last entry; taken from the snapshot if the log is empty.
`truncate_from(i)`	Delete entries from `i` onwards (follower conflict).
`persist(hs, entries)`	Write the hard state (if any) and append/overwrite entries in *one* call; returns `Result`.
`snapshot()`	The current snapshot (returned as a copy).
`install_snapshot(snap)`	Store the snapshot and compact the log prefix; ignores snapshots older than the current one. Returns no error.

14.2 Data model

```
pub struct HardState { pub current_term: Term, pub voted_for: Option<NodeId> }

pub struct Snapshot {
    pub last_included_index: Index, // last entry reflected in state_blob
    pub last_included_term: Term,
    pub conf: Vec<NodeId>, // voter set at snapshot time
    pub state_blob: Vec<u8>, // state machine serialized by the app
}
```

After compaction the log is only a contiguous *suffix* starting at `first_index`. Every entry with index $\leq k$ (`last_included_index`) is considered committed and applied.

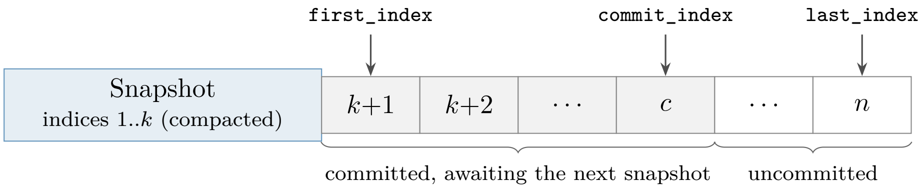

Figure 7: Log layout after compaction: the snapshot replaces prefix $1..k$; the log keeps suffix $k+1..n$.

`install_snapshot` follows Section 7 of the Raft paper [1]: if the log holds an entry at k whose term matches `last_included_term`, the suffix after k is retained; otherwise the whole log is discarded.

14.3 The two existing backends

Data	**`MemoryStorage`**	**`FileSnapshotStorage`**
Hard state	RAM	RAM
Log entries	RAM (`Vec` from `first_index`)	RAM (wraps `MemoryStorage`)
Snapshot	RAM	RAM + file `snapshot.bin` (bincode)
File write	—	Temp file, `fsync`, `rename` (atomic replace)
After crash/restart	Everything lost	Only the snapshot survives; hard state and log tail are lost
Used by	Unit/integration tests	`bin/raft-node.rs` (`DATA_DIR`)

14.4 Local compaction

1. After every apply the engine calls `should_snapshot`: if `last_applied - last_included_index >= snapshot_threshold` it emits `Action::SnapshotNeeded { last_applied }` (the `snapshot_requested` flag prevents repeats).
2. The behaviour turns it into `Event::SnapshotNeeded` for the application.
3. The application serializes its state machine as of `last_applied` and calls `behaviour.compact(index, blob)`.
4. `RaftEngine::compact` checks `0 < index <= last_applied`, looks up the term at `index`, builds `conf` from the current membership, and calls `storage.install_snapshot`.

`raft-node` sets `snapshot_threshold` to 10 000 but has no state machine: on `SnapshotNeeded` it only logs and never calls `compact`.

14.5 Leader → follower snapshot transfer

When `next_index[F] < first_index`, the leader no longer has the entries needed for AppendEntries and switches to `InstallSnapshot`. The `state_blob` is cut into chunks of `DEFAULT_SNAPSHOT_CHUNK_SIZE = 64 KiB`, with only **one** chunk in flight per follower.

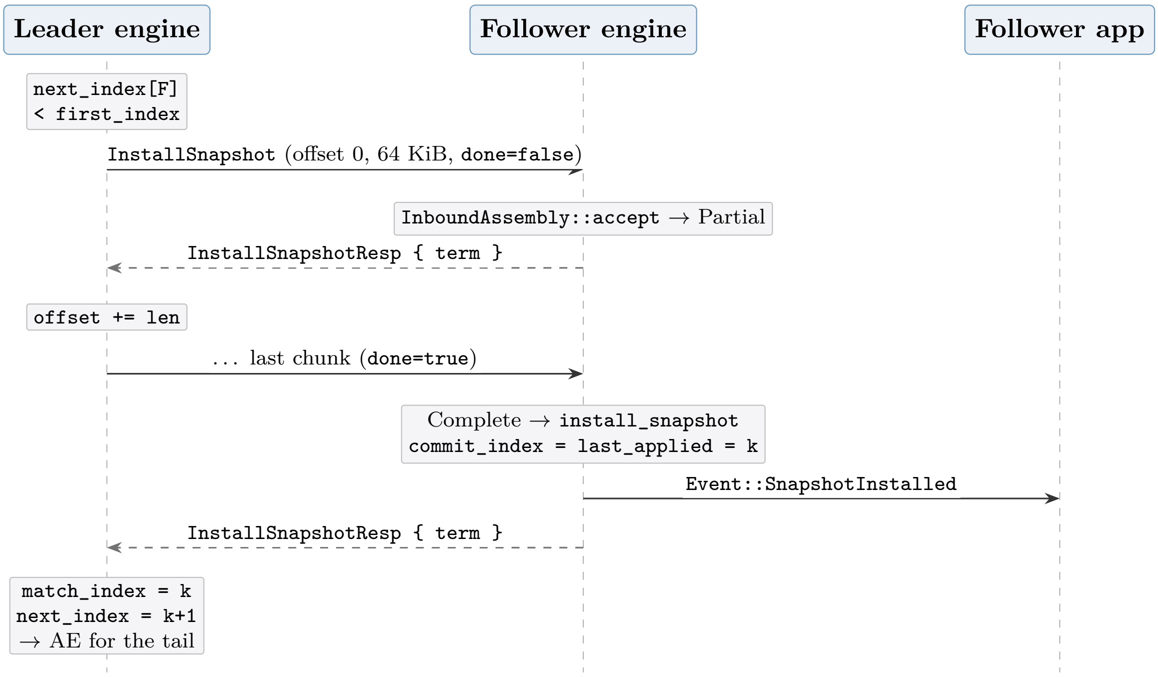
Leader engine
Follower engine
Follower app
next_index[F] < first_index
InstallSnapshot (offset 0, 64 KiB, done=false)
InboundAssembly::accept → Partial
InstallSnapshotResp { term }
offset += len
… last chunk (done=true)
Complete → install_snapshot
commit_index = last_applied = k
Event::SnapshotInstalled
InstallSnapshotResp { term }
match_index = k
next_index = k+1
→ AE for the tail

Figure 8: Chunked snapshot transfer, one chunk at a time (pipeline depth 1).

Current error-handling rules:

- RPC error or timeout → `handle_rpc_failure` deletes `snap_out[F]`; the next attempt restarts from offset 0 (no resume).
- The leader's local snapshot changes mid-transfer → restart from offset 0.
- The follower receives a chunk with `offset = 0` → drops its buffer and starts a new transfer; a chunk with a wrong offset or wrong `(index, term)` → `Rejected`, buffer reset.
- A snapshot with `last_included_index <= commit_index` on the follower → content ignored, a response is still sent.

Throughput: one chunk per round trip, i.e. ≈ `64 KiB / RTT`. At 1 ms RTT that is about 64 MiB/s; at 50 ms RTT only about 1.3 MiB/s, so a 1 GiB snapshot takes about 14 minutes.

14.6 Recovery on startup

1. `FileSnapshotStorage::new(dir)` creates the directory and reads `snapshot.bin` if present; a corrupt file (decode error) returns `StorageError` instead of silently resetting.
2. `RaftEngine::new` sets `commit_index = last_applied = first_index - 1` (everything in the snapshot counts as applied).

3. The hard state is reset to default (term 0, no vote) and the log is empty; the node rejoins as a follower and the leader fills in the tail via AppendEntries or a snapshot.
4. The application must rebuild its state machine from `snapshot.state_blob`; `raft-node` currently only prints the snapshot size.

14.7 Production risks

#	Issue	Consequence
R1	Hard state not durable	A node votes for A in term 5, crashes, restarts with term 0 and then votes for B in term 5 too → two leaders in one term become possible (violates election safety).
R2	Log tail not durable	A follower acknowledges `AppendEntriesResp` success while the entry is only in RAM. If a majority loses power at once, a committed entry (possibly already reported to a client) is lost.
R3	Storage errors are swallowed	The engine calls `let _ = storage.persist(..);` `install_snapshot` returns no `Result` and only logs. The node keeps running with state that diverges from disk.
R4	No `fsync` of the directory after `rename`	On ext4/XFS a power loss right after `rename` can lose the new snapshot.
R5	Snapshot file has no header	No magic, version or checksum; a bincode format change or bit rot cannot be detected or migrated.
R6	Whole snapshot cloned per chunk	`send_snapshot_chunk_to` calls `storage.snapshot()` (which returns a copy) for *every* chunk: a 1 GiB snapshot with 64 KiB chunks ⇒ 16 384 copies of 1 GiB.
R7	Whole blob held in RAM	The leader keeps the blob inside `Snapshot`, the follower assembles it in a `Vec`; state larger than RAM is infeasible.
R8	Response cannot signal rejection	`InstallSnapshotResp` only carries `term`: a `Rejected` chunk is still counted as received by the leader. The system self-heals via a rejected AppendEntries, but wastes a full transfer.
R9	`conf` is not sent on the wire	The follower records `conf` from its own membership, not from the leader's snapshot.
R10	Synchronous serialization	Applications typically call `compact` inside the Swarm loop; a large state blocks the event loop, delays heartbeats and can trigger spurious elections.
R11	Low throughput	One 64 KiB chunk per RTT, no window, no resume.

14.8 Production recommendations

14.8.1 P0 — Durability and safety (mandatory)

- **A truly durable backend** (fixes R1, R2): a segmented append-only WAL, or an embedded KV store (`redb`, `sled`, RocksDB). `persist(hs, entries)` writes *one atomic batch* and `fsync`s before returning.

- **Persist before replying:** persist `voted_for` before sending a granted `RequestVoteResp`; persist entries before sending a successful `AppendEntriesResp`. The engine already calls storage in this order; only the backend needs to be durable.
- **Fail-stop on storage errors** (R3): every write method returns `Result`; on error the engine stops participating in the cluster (surfacing the error so the application shuts down) instead of continuing with non-durable state.
- **Correct file writes** (R4, R5): `fsync` the parent directory after `rename`; give the snapshot file a `magic | version | len | crc32c` header and keep the previous file until the new one is durable.

```
// Proposed: explicit errors + streaming snapshot I/O
pub trait Storage {
    fn persist(&mut self, hs: Option<HardState>, entries: &[LogEntry])
        -> Result<(), StorageError>; // atomic + fsync
    fn truncate_from(&mut self, index: Index) -> Result<(), StorageError>;
    fn snapshot_meta(&self) -> Option<SnapshotMeta>; // index, term, conf, len, crc
    fn read_snapshot_chunk(&self, offset: u64, len: usize)
        -> Result<Vec<u8>, StorageError>;
    fn begin_snapshot_install(&mut self, meta: SnapshotMeta)
        -> Result<Box<dyn SnapshotSink>, StorageError>; // writes to a temp file
}
```

14.8.2 P1 — Non-blocking snapshots and streaming transfer

- **Asynchronous snapshot creation** (R10): the application captures a consistent view at `last_applied` (immutable data structures, copy-on-write, or a DB checkpoint), serializes it in `spawn_blocking`, and only then calls `compact(index, ..)`. The engine keeps running meanwhile.
- **Read chunks from a file** (R6, R7): the leader reads by `offset` from the snapshot file instead of cloning the blob; the follower writes each chunk to a temp file, verifies the checksum, `fsync`s, `rename`s, and only then installs.
- **Extend the wire format** (R8, R9): `InstallSnapshotResp { term, success, next_offset }` for explicit rejection and resume; the first chunk carries `conf`, `total_len` and `crc32c`.
- **Keep some log after the snapshot** (`trailing_logs`): a follower that is only slightly behind still gets AppendEntries rather than a full snapshot.

14.8.3 P2 — Performance and operations

- **Throughput** (R11): a window of N chunks in flight, or a dedicated libp2p substream that streams the whole snapshot with backpressure; chunks of 256 KiB–1 MiB (below `MAX_FRAME_BYTES` = 4 MiB); rate limiting so snapshot traffic does not crowd out heartbeats.
- **A byte-based threshold** in addition to the entry count, so logs with large entries are compacted in time.
- **Metrics:** snapshot size and creation time, bytes transferred, retransmissions, `fsync` latency.
- **Testing:** one `Storage` contract test suite shared by every backend; crash tests (`kill -9` during `persist` and during a snapshot write); Jepsen-style fault injection for simultaneous power loss on a majority of nodes.

Priority order

P0 is the precondition for calling the system crash-safe Raft; without P0 every guarantee holds only if no node ever restarts. P1 is needed once the state machine grows beyond a few tens of MiB. P2 is optimisation for real deployments.

15 Module map of the crate

`lib.rs` exports behaviour, handler, engine, protocol, storage and config. Suggested reading order:

1. `raft/types.rs`
2. `engine.rs` — tick, election, AE
3. `behaviour.rs` — `execute_actions`, `poll`
4. `handler.rs` — unary I/O
5. `storage/`, `raft/snapshot.rs` — persistence and snapshots
6. `bin/raft-node.rs` — Swarm loop

16 Deliberate limitations and status

Implemented: election over libp2p, replication, propose, commit events, chunked snapshot install, a multi-node example, engine tests without a Swarm.

Missing / simplified: AE pipeline > 1; conflict-index optimisation; joint consensus; membership changes; a durable storage backend with an fsync contract (see Section 14); metrics.

Architectural thesis

A pure Raft engine + libp2p Behaviour–Handler + unary multiplexed RPC is a viable composition; the adapter uses correlation IDs, a static peer map, deadline-driven ticks, and separates transport failure from membership.

17 Conclusion

This whitepaper describes in detail how **libp2p-raft** separates consensus from networking: `RaftEngine` upholds the Raft invariants; `RaftBehaviour` translates `Actions` into dials and handler events; `RaftHandler` executes unary framed RPCs over Yamux. The Swarm orchestrates everything without knowing about Raft. Within its simplified scope, the system is a laboratory for studying how a consensus state machine maps onto a modern P2P stack.

References

[1] D. Ongaro and J. Ousterhout, In Search of an Understandable Consensus Algorithm, in *Proc. USENIX ATC*, 2014.

[2] Protocol Labs and contributors, *rust-libp2p* — Swarm, `NetworkBehaviour`, `ConnectionHandler`.